청약 가점표와 커트라인 회귀 정리

2026-10-07 · 당첨 가능성 상/중/하를 더 정확하게 만드는 방법 · 공부 노트

이 문서는 **사용자 가점 계산을 국토부 가점표로 바꾸는 작업**과, **공고별 커트라인을 회귀 모델로 예측하는 아이디어**를 왜 하는지, 둘을 합치면 무엇이 좋은지, 모델이 좋은지 어떻게 판단하는지를 정리한 것이다.

1. 지금 프로젝트 상태와 할 일
2. 국토부 가점표로 바꾸는 이유
3. 상/중/하를 분류·회귀·군집으로 하면?
4. 가점표와 회귀를 엮는 방법
5. 합치면 좋은 점
6. 오차(MAE)와 베이스라인
7. 다음 할 일 체크리스트

1. 지금 프로젝트 상태와 할 일

하던 작업: 가점 계산을 임시식에서 국토부 가점표로 바꾸기

요구사항 정의서 REQ-04의 **"가점은 임시식"** 항목을 해결하는 작업이다. 코드는 거의 끝났다.

파일	바뀐 내용
backend/app/services/eligibility_filter.py	무주택기간·부양가족·통장기간을 각각 함수로 나눠 가점표 구간대로 계산
backend/app/api/routes/recommendations.py	확률 구간 캐시 기준을 '지역'에서 '(지역, 평형)'으로 변경
backend/tests/test_eligibility_filter.py	새 테스트. 구간 경계값 확인. 5개 모두 통과

정리할 것

- **backend/.env.example**이 삭제된 상태. 실수라면 **git restore backend/.env.example**로 되살린다. 팀원들이 이 파일을 보고 .env를 만든다.
- **data_pipeline/_debug_*.py** 4개는 확인용 임시 파일. 확인이 끝났으면 지우고 커밋하지 않는다.
- **models/intent_clf.joblib** 변경. 다시 학습한 거면 커밋, 아니면 되돌린다.

브랜치 만들어 커밋하기

```
git switch -c feat/subscription-score
git add backend/app/services/eligibility_filter.py
git add backend/app/api/routes/recommendations.py
git add backend/tests/test_eligibility_filter.py
git commit -m "[Feat] 청약 가점을 국토부 가점표 구간대로 계산"
```

커밋 후 docs/요구사항_정의서.md의 REQ-04에서 "(가점은 임시식)"을 지운다.

그다음 후보

- REQ-05 대출 매칭에 소득·지역 조건 반영 (loan_matcher.py)
- 클라우드 배포
- RAG 검색을 TF-IDF에서 KURE 임베딩으로 바꾸기
- LangGraph 적용

2. 국토부 가점표로 바꾸는 이유

한마디로 **앱이 보여주는 "당첨 가능성 상/중/하"가 틀리지 않게 하려고**다.

가점이 어디에 쓰이나

> 사용자 정보 → 가점 계산 → 과거 당첨 점수와 비교 → 상/중/하 구간

recommendations.py는 calculate_subscription_score()가 낸 점수를 estimate_percentile_band()에 넘긴다. 그래서 **가점이 틀리면 상/중/하도 틀린다.**

예전 임시식은 얼마나 틀렸나

항목	예전 임시식	실제 가점표	차이
부양가족 0명	0점	5점	기본 5점이 빠져 있었음
무주택 0개월	0점	2점	기본 2점이 빠져 있었음
통장 10년	17점 (만점)	12점	8년 반이면 만점이 돼서 너무 높았음

예: 무주택 3년, 부양가족 0명, 통장 3년인 1인 가구

	무주택	부양가족	통장	합계
예전 임시식	6	0	6	12점
실제 가점표	8	5	5	18점

청약은 1~2점 차이로 당락이 갈리므로 6점 차이는 크다. "하"를 보여줬는데 실제로는 "중"일 수 있다.

가점표 구간 (코드에 반영된 규칙)

항목	규칙	만점
무주택기간	1년 미만 2점, 1년마다 +2점, 15년 이상 32점	32점
부양가족	0명 5점, 1명마다 +5점, 6명 이상 35점	35점
청약통장	6개월 미만 1점, 6개월~1년 2점, 1년마다 +1점, 15년 이상 17점 (통장 없으면 0점)	17점

해야 하는 이유 3가지

- **결과가 정확해진다.** 핵심 기능인 "당첨 가능성 높은 공고 추천"의 기준이 되는 점수다.
- **사용자가 믿고 쓴다.** 사용자는 청약홈에서 자기 가점을 확인할 수 있다. 다르면 다른 결과도 믿지 않는다.
- **면접에서 설명하기 좋다.** "공식 기준표를 따르고, 구간 경계값을 테스트로 검증했다"고 말할 수 있다.

남은 한계: 만 30세 미만 미혼은 무주택기간 점수가 0점이라는 규칙은 아직 반영하지 못했다. 프로필에 나이와 혼인 여부가 없기 때문이다.

3. 상/중/하를 분류·회귀·군집으로 하면?

지금은 규칙 기반 (probability_estimator.py)

```
같은 지역(+평형)의 최근 3년 커트라인 평균을 구한다
내 점수 ≥ 평균 × 1.1  → 상
내 점수 ≥ 평균 × 0.9  → 중
그 외                 → 하
```

문제: **과거 평균을 그대로 쓴다.** 분양가, 세대수, 평형 인기처럼 공고마다 다른 특징을 반영하지 못한다.

우리 데이터의 한계: 개인별 당첨 결과가 없다

historical_results 테이블에 있는 것: 주택형별 cutoff_score(최저 당첨가점), avg_score, top_score, supply_count, competition_rate, region_code, exclusive_area_sqm, year

없는 것: **"가점 X점인 사람이 이 공고에 넣어서 당첨/탈락했다"** 같은 개인 기록. 청약홈은 집계값만 공개한다.

세 방법 비교

방법	무엇을 예측하나	적합도	이유
회귀	새 공고의 커트라인 점수	추천	cutoff_score가 정답 숫자로 DB에 있다
분류	상/중/하 자체	그대로는 어려움	상/중/하 정답 라벨이 없다. 지금 규칙으로 라벨을 만들면 규칙을 따라 할 뿐이다
군집	비슷한 단지끼리 묶기	보조용	정답 없이 묶는 방법이라 상/중/하를 직접 못 정한다. '유사 단지' 찾기에는 쓸 수 있다

회귀로 하면

```
[학습] 과거 주택형 특징(지역, 면적, 연도, 공급세대수, 분양가 ...) → cutoff_score
[예측] 새 공고 특징 → 예상 커트라인 (예: 52.3점)
[판정] 내 점수와 예상 커트라인 비교 → 상/중/하
```

상/중/하 나누는 규칙은 그대로이고, **비교 기준만 "과거 평균"에서 "이 공고 맞춤 예측값"으로 바뀐다.**

데이터 누수 주의: competition_rate(경쟁률)와 apply_count(신청 건수)는 **접수가 끝나야 알 수 있는 값**이다. 추천은 접수 전에 하므로 학습 특징으로 쓰면 안 된다. 쓰면 학습 점수만 좋아 보이고 실제로는 넣을 값이 없다.

군집은 보조로

지금은 "같은 지역이면 비슷한 단지"라고 가정한다. KMeans 등으로 면적·분양가·세대수가 비슷한 단지를 묶으면 더 나은 "유사 단지" 기준이 된다. 회귀 모델이 없을 때의 대체 방법이나 특징 하나로 쓰기 좋다.

시작 전에 확인할 것: 학습 데이터 개수

```
SELECT count(*) FROM historical_results
WHERE result_type = '당첨가점' AND cutoff_score IS NOT NULL;
```

수백 행이면 선형회귀나 랜덤포레스트 정도가 맞다. 너무 적으면 지금 규칙 기반이 더 나을 수도 있다.

4. 가점표와 회귀를 엮는 방법

두 작업은 **상/중/하를 정할 때 비교하는 두 숫자를 각각 맡는다.**

	사용자 쪽	공고 쪽
입력	무주택기간, 부양가족, 통장	지역, 면적, 분양가, 세대수 …
계산	① 가점 계산 (국토부 가점표) eligibility_filter.py	② 커트라인 예측 (회귀 모델) probability_estimator.py
결과	내 점수: 45점	예상 커트라인: 52점
비교	③ 45 < 52 × 0.9 (= 46.8) → "하" (이 부분은 지금 코드 그대로)	

왜 ①을 먼저 끝내야 하나

회귀가 학습하는 cutoff_score는 **청약홈이 공식 가점표로 계산한 실제 당첨자 점수**다. 사용자 점수도 같은 가점표로 계산해야 비교할 수 있다. 눈금이 다른 두 자로 길이를 비교하면 안 되는 것과 같다.

진행 순서

단계	할 일	수정할 곳
1	가점표 작업 마무리, 커밋	eligibility_filter.py, 테스트
2	학습 데이터 개수 확인	DB
3	회귀 실험: 특징 고르기, 학습/검증 나누기, **3년 평균 방식과 오차 비교**	backend/scripts/ 등 새 파일
4	기존보다 오차가 작으면 .joblib로 저장	backend/models/
5	평균 대신 모델 예측값 사용. 모델이 없거나 예측 불가면 기존 평균으로 대체	probability_estimator.py
6	테스트 추가, 요구사항 정의서 갱신	tests/, docs/

backend/models/에는 이미 질문 분류 모델(intent_clf.joblib)을 저장하는 구조가 있다. 같은 방식으로 저장하고 불러오면 된다.

5. 합치면 좋은 점

"이 공고에 내가 넣으면 어느 정도인지"를 공고마다 다르게, 숫자로 설명할 수 있다. 하나만 고치면 안 된다.

예: 실제 가점 18점인 사람이 같은 지역의 두 공고를 볼 때

조합	내 점수	A단지 기준	B단지 기준	결과
임시식 + 3년 평균 (예전)	12점 (틀림)	20점	20점	둘 다 하
가점표 + 3년 평균	18점	20점	20점	둘 다 중
가점표 + 회귀	**18점**	**15점 (외곽, 대형)**	**30점 (역세권, 소형)**	**A 상, B 하**

- 가점표만: 내 점수는 맞지만 같은 지역 공고는 모두 같은 결과.
- 회귀만: 공고별 기준은 생기지만 내 점수가 틀려 비교도 틀림.
- 둘 다: 같은 지역 안에서도 "A는 해볼 만, B는 어렵다"를 구분.

좋은 점 4가지

- **추천 순서가 의미 있어진다.** 넣어볼 만한 공고가 위로 올라온다.
- **"몇 점 차이"까지 보여줄 수 있다.** 예: 내 점수 18점 · 예상 커트라인 30점 · 12점 부족
- **할 일을 알려줄 수 있다.** 예: 통장 1년 더 유지하면 +1점, 무주택 1년 더 유지하면 +2점
- **면접에서 설명하기 좋다.** 정확해야 하는 부분은 규칙으로, 불확실한 미래 값은 모델로 나눠 결합했다.

	맡는 부분	방식
가점표	내 점수 (정답이 정해져 있음)	규칙
회귀	이 공고 커트라인 (미래라 모름)	예측

정답이 있는 건 규칙으로 정확하게, 모르는 건 데이터로 예측한다.

6. 오차(MAE)와 베이스라인

정확도가 올라간다고 말할 수 있는 근거

- **가점표:** 확실히 올라간다. 공식 기준표를 따랐고 경계값 테스트로 확인했다.
- **회귀:** 올라갈 것으로 **기대**하는 단계. 데이터가 적거나 특징이 약하면 3년 평균보다 못할 수도 있다. 그래서 숫자로 비교해야 한다.

오차란: 실제 값과 얼마나 다르게 맞혔나

검증용 공고 3개 예시 (실제 커트라인은 이미 아는 과거 데이터)

공고	실제	3년 평균 예측	틀린 정도	회귀 예측	틀린 정도
A	45	52	7	47	2
B	60	52	8	56	4
C	50	52	2	49	1
평균			**5.7점**		**2.3점**

이 "평균적으로 몇 점 틀렸나"가 **MAE(평균 절대 오차)**다. 점수가 아니라 오차라서 **낮을수록 좋다.** 회귀가 더 작으면 실제에 더 가깝게 맞혔다는 뜻이므로 바꾼다.

회귀가 더 크면 바꾸지 않는다

모델은 공짜가 아니다.

- 모델 파일을 관리해야 한다
- 데이터가 쌓이면 다시 학습해야 한다
- "왜 52점으로 예측했나"를 설명하기 어려워진다

이 비용을 감수할 만큼 정확할 때만 바꾼다. 지금 쓰는 3년 평균처럼 단순한 기준을 **베이스라인(baseline)**이라 하고, 새 모델은 항상 베이스라인을 이겨야 쓸 이유가 생긴다.

> **검증은 학습하지 않은 데이터로.** 학습한 데이터로 채점하면 답을 보고 시험 본 것과 같아서 오차가 실제보다 작게 나온다.

몇 점이면 좋은가: 0에 가까울수록

0점이면 모든 공고를 정확히 맞혔다는 뜻이지만 현실에서는 거의 나오지 않는다. 그래서 세 가지를 같이 본다.

기준	설명
0에 가까울수록 좋다	MAE는 틀린 정도라서 작을수록 좋다
베이스라인보다 작아야 한다	3년 평균이 5.7점이면 회귀가 4점만 나와도 개선 (가장 중요)
문제에 비해 충분히 작아야 한다	청약은 1~2점 차이로 당락이 갈린다. 10점이면 상/중/하가 자주 뒤집혀 쓰기 어렵고, 2~3점이면 꽤 믿을 만하다

7. 다음 할 일 체크리스트

- [ ] backend/.env.example 복구 (실수로 지운 경우)
- [ ] _debug_*.py 4개 정리, intent_clf.joblib 변경 확인
- [ ] feat/subscription-score 브랜치 만들어 가점표 작업 커밋
- [ ] 요구사항 정의서 REQ-04에서 "(가점은 임시식)" 지우기
- [ ] DB 켜고 커트라인 데이터 개수 확인 (3장 SQL)
- [ ] 회귀 실험: 누수 특징 빼기, 학습/검증 나누기, 3년 평균과 MAE 비교
- [ ] 이기면 모델 저장 후 probability_estimator.py에 연결 (fallback 유지)